

Interpreter and Compiler Modes for LLM-Assisted Workflows: Compiling an Exploratory Agent into a Production Market-Digest Pipeline

aobathree

Independent researcher · Prepared with the assistance of ChatGPT, Claude, and Cursor · August 2, 2026

Artifacts: <https://github.com/aobathree/lm-interpreter-compiler-modes>

Abstract—LLM agents excel at discovering procedures for unfamiliar tasks but are slow, costly, and non-deterministic when the same procedure is repeated. We articulate a two-phase model separating *exploration* (the LLM discovers a procedure; *interpreter mode*) from *execution* (the procedure is fixed as deterministic code; *compiler mode*), connected by a guard-triggered fallback loop analogous to JIT deoptimization. On a recurring cryptocurrency market-digest task, compilation reduced LLM context consumption from tens of thousands of tokens to under ten lines per run; successive stages reduced latency from minutes to ~ 0.1 s, and process-tree accounting revealed a $42\times$ CPU reduction invisible to wall-clock measurement. Scaling to 62 instruments surfaced server-side rate limiting, absorbed by a concurrency guard. The pipeline now runs hourly on serverless infrastructure at negligible cost, and the design was proposed upstream to the vendor’s CLI project.

1 Introduction

LLM agents interleave reasoning with tool invocation, which suits tasks whose procedure is unknown in advance. Once the procedure stabilizes, however, re-running the agent repeats inference cost on every execution: contexts fill with raw intermediate data, latency is dominated by model calls, and outputs are irreproducible. We treat agent operation as two modes: in *interpreter mode* the LLM explores to discover a working procedure; in *compiler mode* that procedure is fixed as deterministic code with no LLM on the critical path. A guard-and-fallback loop returns control to the LLM only when compiled assumptions break, after which the procedure is recompiled. Because fixing occurs only after observing real executions, the model resembles JIT compilation with deoptimization rather than ahead-of-time compilation.

This position aligns with the industry shift toward executing code rather than chaining model-mediated tool calls [3,4] and with research on distilling agent trajectories into reusable workflows [1,2,5]. Our contribution is an end-to-end case study covering the full lifecycle—exploration, compilation, measurement-driven hardening, production deployment, and upstreaming—quantifying effects that partial treatments omit, notably resource costs hidden behind unchanged wall-clock time.

2 The Two-Mode Model

An agent trajectory is compiled by decomposing it into three parts: (i) operations that can be fully fixed (data retrieval, aggregation, formatting), which become ordinary code; (ii) values that must remain variable (e.g., the target instrument), which become parameters; and (iii) residual semantic judgment (e.g., "comment briefly on the digest"), which remains a small LLM invocation. The optimal result is therefore rarely an LLM-free script; it is a deterministic skeleton with narrowly scoped LLM calls and an agent fallback path. Naïve record-and-replay fails in practice—incidental retries are misinterpreted as procedure, values are hard-coded, and implicit decision criteria are lost—so compilation additionally requires input identification, idempotency awareness, guard conditions, and golden tests that compare compiled output against the interpreted run.

3 Case Study: A Recurring Market Digest

The task is a recurring trading-condition digest (price, RSI, MACD sign, moving-average position, volume pattern, ATR) for pairs listed on the bitbank cryptocurrency exchange. In interpreter mode, the LLM fetched hundreds of raw OHLCV candles per pair and computed indicators in-context—tens of thousands of tokens per run, minute-scale latency, arithmetic risk. The stabilized procedure was compiled into a Python script emitting a three-line digest per pair; the LLM thereafter reads only the digest. Two further stages followed: a Rust reimplementing parallelizing all fetches, and a variant calling the exchange REST API directly instead of spawning the vendor CLI as subprocesses. Output equivalence across implementations was verified byte-for-byte (golden testing); Table 1 summarizes the latency ladder.

Stage	Implementation	LLM reads	Latency
0	LLM in-context computation	$>10^4$ tokens	minutes
1	Python script (sequential)	~ 10 lines	0.6 s
2	Rust, parallel fetch	~ 10 lines	0.11 s
3	Rust, direct REST API	~ 10 lines	0.11 s

Table 1. Compilation stages (3 pairs; medians). Stages 1–3 produce byte-identical digests.

The stage-1 $\rightarrow$ 2 speedup ($5.5\times$) is attributable almost entirely to eliminating interpreter startup and parallelizing six network fetches, not to language choice: indicator computation is sub-millisecond in both implementations, and the workload is network-bound.

4 Scale Surfaces Hidden Constraints

Extending from 3 to all 62 listed pairs (124 parallel fetches; >300 HTTP requests) exposed a constraint invisible at small scale: with unbounded parallelism, runs above ~ 30 pairs intermittently stalled for 15–26 s without any error being returned, a pattern consistent with token-bucket rate limiting. Introducing a concurrency guard (a semaphore capping in-flight requests at 16) restored stable behavior: all 62 pairs complete in ~ 1.2 s, matching the estimate $124 \text{ fetches} \div 16 \approx 8 \text{ waves} \times 0.15 \text{ s}$. This episode instantiates the model’s fallback loop in practice: a compiled assumption (unbounded parallelism is safe) failed under scale, was diagnosed, and the procedure was recompiled with a guard.

5 Wall-Clock Parity Can Hide a $42\times$ Resource Gap

Stage 3 initially appeared to offer only $\sim 1.2\times$ wall-clock improvement over stage 2, inviting the conclusion that removing a call layer has little value. Process-tree accounting (Windows Job Objects, capturing all short-lived children) showed otherwise (Table 2): the CLI-subprocess design spends $42\times$ more CPU and two orders of magnitude more processes and page faults, hidden by network-bound waiting on a multi-core host. On CPU-constrained targets the difference re-emerges as wall-clock time: “not faster” and “not lighter” are distinct claims.

Metric (62 pairs)	CLI subproc.	Direct API	Ratio
Wall-clock time	1.1–1.7 s	1.1 s	$\sim 1.2\times$
CPU time (tree)	1.97 s	0.05 s	$42\times$
Processes	125	1	$125\times$
Page faults	3.2×10^5	6.5×10^3	$50\times$
Peak memory (job)	30 MB	20 MB	$1.5\times$

Table 2. Resource consumption via Job Object accounting (medians of 3 runs).

6 Deployment and Upstreaming

The measured lightness (0.05 s CPU, 20 MB, one process) permits production operation on minimal serverless infrastructure: an hourly schedule invokes a 256 MB arm64 Rust Lambda function that reads its target-pair configuration from a single DynamoDB record, generates the digest, and posts it to a Discord webhook chunked to the 2,000-character limit. Configuration changes made through a static single-page GUI take effect at the next run without redeployment; operating cost is effectively zero. The decomposition of Section 2 also indicates where each component should live—deterministic computation in the vendor’s CLI, the trigger/commentary layer in a thin agent skill—and we proposed a periodical-brief subcommand upstream with the reference implementation and measurements [6].

7 Conclusion

The interpreter/compiler framing is practical and compounds: 10^3 -fold context reduction from compilation itself, order-of-magnitude latency and resource gains from classical optimization that determinism makes safe to apply, and a near-zero-cost production path. Two lessons generalize: hypotheses about where time is spent do not survive measurement, and scale breaks compiled assumptions in ways only guards and re-exploration absorb. The full lifecycle—explore, compile, measure, guard, operate, upstream—is itself the unit of value.

References

- [1] Z. Wang et al., "Agent Workflow Memory," arXiv:2409.07429, 2024.
- [2] S. Kim et al., "An LLM Compiler for Parallel Function Calling," arXiv:2312.04511, 2023.
- [3] Cloudflare, "Code Mode: the better way to use MCP," Cloudflare Blog, Sep. 2025.
- [4] Anthropic, "Code execution with MCP," Anthropic Engineering Blog, Nov. 2025.
- [5] O. Khattab et al., "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines," arXiv:2310.03714, 2023.
- [6] aobathree, "[feat] periodical-brief," bitbankinc/bitbank-lab-cli, Issue #21, Aug. 2026. <https://github.com/bitbankinc/bitbank-lab-cli/issues/21>